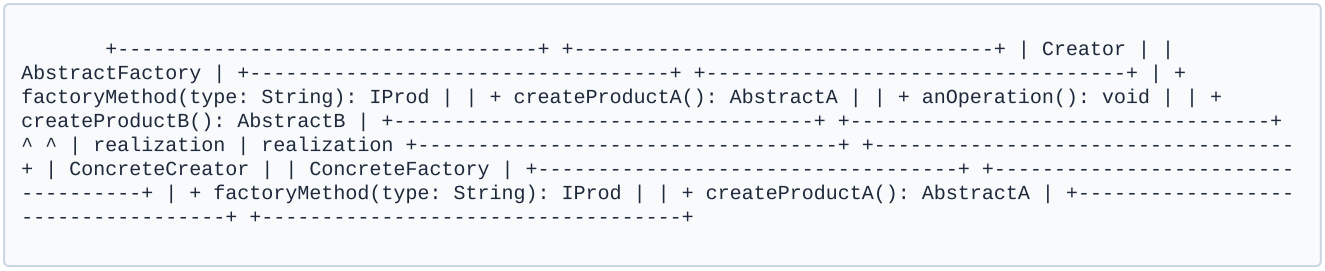
Detailed Design Patterns & UML Specifications

Featuring Object-Oriented Structures, Signatures, and Core Data Types

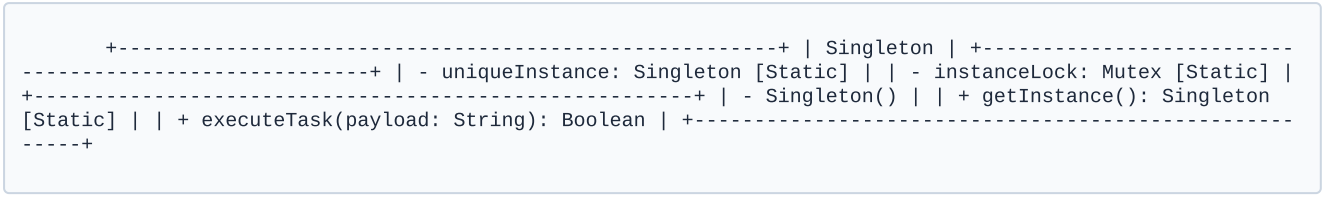
1. Factory Method & Abstract Factory (Creational)

Factory defines creation contracts; Abstract Factory manages families of related products with explicit return types.



2. Singleton (Creational)

Guarantees a single instance globally via private constructors and static reference fields with thread-safety guards.



3. Adapter Pattern (Structural)

Bridges incompatible interfaces by translating client target requests into adaptee native method calls.

```
      +-----------------------+ +--------------------------------+ | Client |-----> | <<interface>>
Target | +-----------------------+ +--------------------------------+ | + request(data: Map): Response |
+--------------------------------+ ^ | realization +--------------------------------+ | Adapter | +-----
---------------------------+ | - adaptee: Adaptee | | + request(data: Map): Response | --->
[Adaptee].specificOp(xml: String)
```

4. Composite Pattern (Structural)

Builds recursive tree hierarchies allowing uniform treatment of primitive leaves and container composites.

```
      +--------------------------------+ +--------------------------------+ | <<interface>> Component |
| Leaf | +--------------------------------+ +--------------------------------+ | + operation(): void |<-
------| + operation(): void | +--------------------------------+ +--------------------------------+ ^ |
aggregation +--------------------------------+ | Composite | +--------------------------------+ | -
children: List<Component> | | + add(c: Component): void | | + operation(): void | +----------------------
----------+
```

5. Facade Pattern (Structural)

Simplifies interaction with deep subsystem layers by routing client requests through a clean unified entry point.

```
      +-----------------------+ +-----------------------------------+ | Client |------>| Facade | +----
-------------------+ +-----------------------------------+ | - subA: SubsystemA | | - subB: SubsystemB |
+-----------------------------------+ | + runWorkflow(config: Config): Int | +--------------------------
---------+
```

6. Command Pattern (Behavioral)

Encapsulates requests as parameterizable objects to support queuing, logging, and undo/redo workflows.

```
      +-----------------------+ +--------------------------------+ | Invoker |-----><<interface>>
Command | +-----------------------+ +--------------------------------+ | - history: Stack<Cmd> | | +
execute(): void | | + invoke(c: Command) | | + undo(): void | +-----------------------+ +---------------
-----------------+ ^ | realization +--------------------------------+ | ConcreteCommand | +-------------
------------------+ | - receiver: Receiver | | + execute(): void | ---> [Receiver].action()
```

7. Memento Pattern (Behavioral)

Captures internal object states externally for safe restoration without violating structural encapsulation bounds.

```
      +--------------------------------+ +--------------------------------+ | Originator | | Memento |
+--------------------------------+ +--------------------------------+ | - state: StateData | | -
savedState: StateData | | + saveState(): Memento |---->| - Memento(state: StateData) | | + restoreState(m:
Memento) | | ~ getState(): StateData | +--------------------------------+ +----------------------------
---+
```

8. Strategy Pattern (Behavioral)

Encapsulates interchangeable algorithmic families behind a uniform interface resolved dynamically at runtime.

```
      +--------------------------------+ +--------------------------------+ | Context | | <<interface>>
Strategy | +--------------------------------+ +--------------------------------+ | - strategy: Strategy
|---->| + execute(data: List): Result | | + setStrategy(s: Strategy) | +--------------------------------+
| + executeContext(): Result | ^ +--------------------------------+ | realization +----------------------
-----------+ | ConcreteStrategy | +--------------------------------+ | + execute(data: List): Result | +-
--------------------------------+
```